Machine Learning with Applications 17 (2024) 100578



Contents lists available at ScienceDirect

Machine Learning with Applications

journal homepage: www.elsevier.com/locate/mlwa





A framework for modelling customer invoice payment predictions

Willem Roux Moore *, Jan H. van Vuuren

Stellenbosch Unit for Operations Research in Engineering, Department of Industrial Engineering, Stellenbosch University, Private Bag XI, Matieland, 7602, South Africa

ARTICLE INFO

Keywords:
Machine learning
Survival analysis
Survival boost
Decision support systems
Invoice payment
Payment prediction

ABSTRACT

By offering clients attractive credit terms on sales, a company may increase its turnover, but granting credit also incurs the cost of money tied up in *accounts receivable* (AR), increased administration and a heightened probability of incurring bad debt. The management of credit sales, although eminently important to any business, is often performed manually, which may be time-consuming, expensive and inaccurate. Such an administrative workload becomes increasingly cumbersome as the number of credit sales increases. As a result, a new approach towards proactively identifying invoices from AR accounts that are likely to be paid late, or not at all, has recently been proposed in the literature, with the aim of employing intervention strategies more effectively. Several computational techniques from the credit scoring literature and particularly techniques from the realms of survival analysis or machine learning have been embedded in the aforementioned approach. This body of work is, however, lacking due to the limited guidance provided during the data preparation phase of the model development process and because survival analytic and machine learning techniques have not yet been ensembled. In this paper, we propose a generic framework for modelling invoice payment predictions with the aim of facilitating the process of preparing transaction data for analysis, generating relevant features from past customer behaviours, and selecting and ensembling suitable models for predicting the time to payment associated with invoices. We also introduce a new sequential ensembling approach, called the Survival Boost algorithm. The rationale behind this method is that features generated by a survival analytic model can enhance the efficacy of a machine learning classification algorithm.

1. Introduction

Companies typically generate sales revenue through two channels, namely cash received and credit granted. Cash transactions are usually preferred as it increases cash on hand immediately. In certain cases, however, credit transactions are preferred or required by customers and are then managed as *accounts receivable* (AR). AR, as defined in accounting, represent funds that are owed by customers to an entity in exchange for goods or services that have already been delivered or used, but for which payment has not yet been received (Peiguang, 2015).

Apart from a select few retailers or small 'high street' shops, most companies offer credit sales to their customers, thereby providing a financial service as well as their basic services or products. Such credit terms are sought by customers for several reasons, including that they might not have cash on hand at the time of purchasing services or products, and/or that they may earn interest on cash held back during the period of credit extension (Firth, 1976). By offering clients attractive credit terms on sales, a company may increase its turnover, but granting credit also incurs the cost of money tied up in AR, increased administration and a heightened probability of incurring bad debt. Hence there is a trade-off between increased profits earned from sales on credit, on the one hand, and the cost and risk associated with managing such credit, on the other (Firth, 1976).

In order to manage the risk emanating from this trade-off, a set of intervention activities, commonly referred to as the invoice-to-cash process, is required. These activities are aimed at ensuring that cash is generated from sales and include the prioritisation of certain accounts, contacting customers, placing collection calls, the escalation of accounts and the resolution of disputes. Although eminently important to any business, this process is often performed manually, which may be time-consuming, expensive and inaccurate (Cheong & Shi, 2018; Zeng et al., 2008). Moreover, the administrative workload associated with these activities becomes increasingly cumbersome as the number of credit sales increases.

* Corresponding author.
E-mail addresses: 19993293@sun.ac.za (W.R. Moore), vuuren@sun.ac.za (J.H. van Vuuren).

[1] *Survival analysis* is the statistical theory underlying analyses of the duration of time until an event of interest is expected to occur, such as the death of a biological organism or the failure of a mechanical system (Appel et al., 2019).

https://doi.org/10.1016/j.mlwa.2024.100578
Received 28 March 2024; Received in revised form 25 June 2024; Accepted 18 July 2024
Available online 19 July 2024
2666-8270/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

The problem described above gave rise to a new body of research during the 1970s, dedicated to the development of techniques for proactively identifying invoices from AR that are likely to face late payment or no payment at all. The underlying problem, formally known as the *invoice payment prediction problem* (IPPP), has traditionally been tackled by invoking statistical techniques from the realm of survival analysis.[1] These techniques have been applied in a wide variety of areas, including analyses of the credit data of financial institutions (Dirick et al., 2017) and modelling consumer credit risk (Cao et al., 2009; Cheong & Shi, 2018; Rychnovsky, 2018). While survival analysis methods excel in predicting *when* an event is likely to occur, such as the event of settling an account, they are less effective in predicting *whether* that event will occur at all. To address this problem, the IPPP has also been modelled as a classification problem, in which invoices are classified according to classes representing the expected time to payment.

While an abundance of research has been dedicated to the development of models for customer credit scoring, there is only a small body of research on how to apply these models to the IPPP. With the introduction of new accounting standards by the *International Federation of Reporting Standards* (IFRS), business entities are required to recognise a financial asset at its fair value, thereby resulting in a situation where older AR are adjusted proportionally to recognise the risk of non-payment (PWC, 2018). In the case of invoices, the asset value should be adjusted based on the number of days over which the invoice remains unpaid. This allows for the accurate quantification of the time duration over which an invoice is left unpaid if all payments from an account are allocated to the oldest invoice first. The existing literature, however, does not make provision for such adjustments, as these adjustments are mainly applied to data sets in which payments are associated with *specific* invoices.

Moreover, many of the techniques for predicting invoice payments proposed in the literature adopt either a survival analytic approach or a machine learning approach, while the relative performances of models from both realms are compared in rare cases only. There is, however, to the best of our knowledge, no work in which the performance of models from both these realms are ensembled in a sequential fashion. In this paper, we propose a new approach towards resolving instances of the IPPP, called the *Survival Boost* algorithm, which is based on the notion that survival analytic models may enhance or 'boost' the performance of machine learning classification models when information is passed from the survival analytic paradigm to the machine learning paradigm in the form of additional features.

Moreover, the two shortcomings in the literature described above have given rise to the need for a framework in support of the evaluation of AR that facilitates the process of modelling invoice payment predictions. We therefore also propose a *generic* framework, referred as the *Modelling Invoice Payment Predictions* (MIPP) framework, for facilitating the processes of preparing transaction data for analysis, generating relevant features based on past customer behaviour, and selecting and ensembling suitable models from the realms of survival analysis and machine learning for predicting the time to payment associated with invoices. Furthermore, the analysis of model results is strengthened by the identification of useful trends and patterns in support of high-quality decision making on the part of a user of the framework. The framework may be absorbed into a computerised *decision support system* (DSS) that enables companies to leverage their structured transaction data in combination with other relevant data sources in support of decision making, adjusting business processes, and improving collection efforts.

The remainder of this paper is structured as follows. An overview of the relevant literature is presented in Section 2, detailing existing approaches towards solving the IPPP, together with their respective shortcomings. The proposed MIPP framework is presented next in Section 3 where a discussion on the working of the novel *Survival Boost* algorithm is proffered. The practical value of the framework is demonstrated thereafter by applying it to a real-world case study in Section 4. The paper finally closes in Section 5 with a brief discussion and appraisal of its contributions, as well as proposals for follow-up future work.

2. Literature review

This section is devoted to a brief review of the literature pertaining to machine learning, survival analysis, and ensemble methods for resolving instances of the IPPP in Section 2.1, Section 2.2, and Section 2.3, respectively. The review finally closes in Section 2.4 with the presentation of a case for the development of a generic framework that is capable of guiding research on the resolution on IPPP instances.

2.1. Survival analysis methods

Survival analysis is the analysis of time-to-event data that describe an expected duration of interest over some time horizon from a specified time origin to a specified endpoint in time. The applications of survival analysis in the contemporary medical literature are vast and widespread, especially in the realms of clinical and epidemiological research (Flynn, 2012).

One of the primary goals of survival analysis is to determine the so-called *survival function*, which represents the probability that an event of interest, at time T, pertaining to some group of subjects, will not occur before a specific time t (Klein, 2005; Wang, 2005). This function is given by

$$S(t) = P(t \geq T), \tag{1}$$

where $P(\cdot)$ denotes the probability of an event $\cdot$ occurring. The survival function is a monotonically decreasing function of time t and initially assumes the value 1 at time $t = 0$, which indicates that no event of interest has occurred at the start of the time period under consideration and hence that all of the subjects have survived up to that point in time. The *cumulative death distribution function* (CDDF) is defined as

$$F(t) = 1 - S(t) \tag{2}$$

and represents the probability that the event of interest occurs earlier than at time t. Furthermore, the *death density function* (DDF), denoted by $f(t)$, is obtained by computing the instantaneous rate of change of the CDDF with respect to time. That is,

$$f(t) = \frac{\mathrm{d}}{\mathrm{d}t} F(t) \tag{3}$$

in the continuous case, or

$$f(t) = \frac{F(t + \Delta t) - F(t)}{\Delta t} \tag{4}$$

in the discrete case, where Δt denotes some small time interval in the latter case.

Many survival analytic models available in the literature for solving the IPPP have emanated from the related domain of credit scoring. The objective in credit scoring is to determine *whether* credit applicants will be 'good' or 'poor' payers, depending on their recent repayment histories. The techniques traditionally used to answer this question include, among others, discriminant analysis (Durand, 1941), logistic regression (Henley, 1995), and genetic algorithms (Abdou, 2009).

If one considers an applicant's credit worthiness as a dynamic attribute instead of a static attribute, the underlying objective pursued changes over time. It is, therefore, no longer of interest *whether* an applicant will default on his or her loan, but rather *when* applicants will likely default, given that they are poor payers (Banasik et al., 1999). This complicates matters, since the time to default is a continuous value and the exact time to default is often not known for all customers in a sample due to the possibility of censored data.

In order to address these difficulties, Narain (1992) first introduced the notion of *accelerated failure time* (AFT) survival analytic models as an alternative to the aforementioned IPPP solution approaches. Banasik et al. (1999) followed Narain's example of applying survival analysis techniques, but instead proposed the use of Cox proportional hazard models (Cox, 1972) in an attempt to model time to default. In their

study, Banasik et al. concluded that the performance of Cox proportional hazard models is competitive with that of logistic regression. Subsequent work in this domain extended AFT and Cox proportional hazard models by incorporating coarse classification (Stepanova & Thomas, 2001) and time-varying covariates (Bellotti & Crook, 2009). Furthermore, Dirick et al. (2017) presented a benchmark study of various survival analytic techniques used in credit scoring, by applying them to data sets obtained from financial institutions in Belgium and Great Britain.

Smirnov (2016) studied the application of survival analytic models to instances of the IPPP and introduced the notion of *random survival forests* (RSF) to estimate payment probability when applied to right-censored invoice data. In his thesis, Smirnov concluded that in conjunction with historical payment data, RSF outperformed traditional Cox proportional hazards models.

In a related paper, Nanda (2018) studied the problem of reducing AR by implementing an improved collection strategy. The estimated time to payment was modelled as a continuous variable and various regression algorithms were used to estimate the likely payment date. Performance in this case was measured as the total number of examples of an algorithm predicting the exact date of payment correctly.

2.2. Machine learning methods

The IPPP has also been modelled as a classification problem, although there is only small body of work in the literature that addresses the problem in this form. One of the first papers adopting this approach was by Zeng et al. (2008), in which the problem was formulated as a supervised machine learning classification problem, with the classes representing predicted payment times. The three classes 1–30 days, 31–60 days, and 61+ days were considered. It was found that adding historical predictors improved the performance of the models. In a related study, Peiguang (2015) formulated the IPPP as a binary prediction problem in which invoices were to be classified as being either paid late or paid on time.

Several other studies followed a similar approach to that adopted by Peiguang (2015), in which invoices were classified as paid on time or late (Appel et al., 2019; Cheong & Shi, 2018; Nanda, 2018). Appel et al. (2019) presented a prototype to support collection agents by predicting whether invoices will be paid within five days or not. The authors concluded that variable window sizes dramatically influence the performance of the classification model when aggregating a customer's recent payment history. In general, it was found that more emphasis should be placed on recent payment behaviour than on earlier behaviour so as to improve model accuracy.

Tater et al. (2018) studied a variation of the IPPP, focusing only on *accounts payable* (AP) instead of AR. AP refers to invoices issued by a vendor to a company which are payable within a stipulated timeframe. In this context, the authors modelled the problem as a supervised learning problem and predicted whether or not a company was likely to pay an invoice late.

Bahrami et al. (2020) introduced a novel behavioural scoring model which was used as input to their predictive models. The authors studied Dunning actions (the set of actions performed by a service provider company when customers have unpaid invoices). These actions are performed with the aim of reducing the time interval over which invoices remain unpaid. Examples of such actions include *short message service* (SMS) or phone calls threatening of legal action. The authors developed a behavioural scoring model based on Dunning actions and the reactions of customers to these actions. The behavioural score was subsequently employed during a feature engineering process so as to improve upon the overall performance of their prediction model(s). As in the studies mentioned earlier (Appel et al., 2019; Cheong & Shi, 2018; Nanda, 2018; Peiguang, 2015), Bahrami et al. (2020) modelled the IPPP as a binary classification problem in which invoices were classified as being paid late or on time, based on whether or not the full amount outstanding was paid before the due date.

Schoonbee et al. (2022) presented a machine learning roadmap embedded in a decision support system with a view to support companies in their prioritisation of debt collection as part of the IPPP. The system is capable of predicting payment behaviour outcomes linked to invoices, such that payment receipts are categorised into one of three intervals: 1–30 days late, 31–60 days late, or at least 61 days late. The working of the system was demonstrated in the form of a case study. The main takeaway of the study was that a random forest algorithm outperformed the other algorithms considered, achieving an average accuracy of 83.61%.

In his master's thesis, Martikainen (2023) employed various machine learning classification algorithms to predict late payments of customers. The contribution of the study was the development of a novel classifier for predicting the likelihood of invoice payment, based entirely on the actions performed in respect of previous invoices issued to the same customer, applying exponential discounting to emphasise more recent behaviour. The key finding of the study was that recent payment behaviour is the most significant contributor to predictions.

2.3. Ensemble methods

In general, the architecture of an ensemble of classification learning models comprises several *base learners*, an input vector $\boldsymbol{x}$, and some function for combining the outputs from the various base learners in order to arrive at a collective decision $\boldsymbol{y}$. A base learner in this context refers to a learning algorithm that is trained in respect of a training data set (Zhou, 2012). In mathematical terms, the prediction of an ensemble of learning models may be expressed as

$$\varphi(\boldsymbol{x}_i) = G\left(f_1(\boldsymbol{x}_i), f_2(\boldsymbol{x}_i), \ldots, f_k(\boldsymbol{x}_i)\right) = \hat{\boldsymbol{y}}_i, \tag{5}$$

where G denotes some aggregation function, the predictions of the base learners are denoted by $f_1, f_2, \ldots, f_k$ and $\hat{\boldsymbol{y}}_i$ denotes the single prediction based on the input vector $\boldsymbol{x}_i$. When selecting base learners, it is possible to create a *homogeneous* ensemble in which several base learners of a single type are used. Alternatively, a *heterogeneous* ensemble, or *multiple classifier system*, can be created using various different types of learning models (Rokach, 2010; Zhou, 2012).

There is, to the best of our knowledge, only one example of a heterogeneous ensemble in the IPPP literature. This example may be found in the work of Smirnov (2016), who adopted RSF to predict invoice payments. This algorithm can be described as an extension of the well-known random forest algorithm, in which the splitting criteria of the various decision trees comprising the forest are based on some survival splitting criterion, *i.e.* nodes are split so as to maximise the survival differences across child nodes. The information obtained from the various trees are then combined using an ensemble cumulative hazard estimate, thereby leading to an aggregated prediction (Smirnov, 2016).

There are, however, many instances in the IPPP literature in which homogeneous ensembles have been created, including many instances in which random forest, XGBoost, and other gradient boosting algorithms have been invoked (Appel et al., 2019; Bahrami et al., 2020; Cheong & Shi, 2018; Martikainen, 2023; Nanda, 2018; Peiguang, 2015; Zeng et al., 2008).

2.4. A case for a new generic framework

In summary, the existing literature related to the IPPP is largely focused on applying various predictive modelling techniques to a particular case study and comparing the resulting performance with that achieved by other methodologies. Very little guidance is, however, available on how to apply these models in practice, based on the adoption of a generic and *holistic* framework which encapsulates data preprocessing, debtors' ageing analyses, and summarisation. Moreover, the

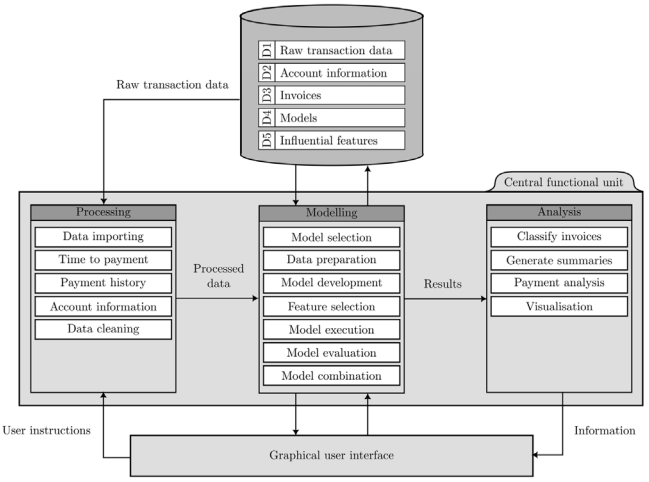


Fig. 1. A high-level schematic representation of the MIPP framework.

existing research provides little guidance on how to create a heterogeneous ensemble of survival models and machine learning classification models. This is identified as a serious shortcoming, since it is anticipated that more accurate and robust results may be achieved by an ensemble of the diverse methodologies of survival models and machine learning models, than may be achieved by one of these methodologies when considered in isolation.

The framework proposed in this paper differs from those in the literature in the following areas:

1. The framework *guides* a user through the process of preparing transaction data facilitating a resolution of the IPPP in the case of reoccurring invoices, where customer payments are allocated to oldest debt first.
2. Rather than embedding a select few models in the framework, guidance is provided in the form of a *model development process*, during which model selection, feature engineering, feature selection, and results presentation are required.
3. The framework is designed to be *generic* and *flexible*, so that it is applicable in any sector and so that it is configurable according to the subjective preferences of a user.
4. The framework facilitates the development of a heterogeneous ensemble of a survival model and a machine learning classification model that is applicable to any IPPP instance.

3. Proposed invoice payment prediction framework

In the literature review presented in the previous section, we outlined the need for a generic framework that facilitates the intricate and complex resolution of the IPPP. This section is dedicated to a detailed presentation of our MIPP framework designed for this purpose. A high-level overview of the framework is presented first, after which the constituent components of the framework are described in conjunction with *data flow diagrams* (DFDs). The MIPP framework is designed to be generic and modular in nature so as to ensure its applicability in various domains. Modules included in the framework may thus be deleted, modified, or substituted, depending on the application of the framework to a particular domain. This characteristic of the framework is particularly useful in the context of the IPPP, where modelling approaches from various paradigms have often been employed.

A high-level schematic representation of the MIPP framework may be found in Fig. 1. The cornerstone of the framework is the database component, in which relevant input data are stored throughout the execution of the various processes of the central functional component.

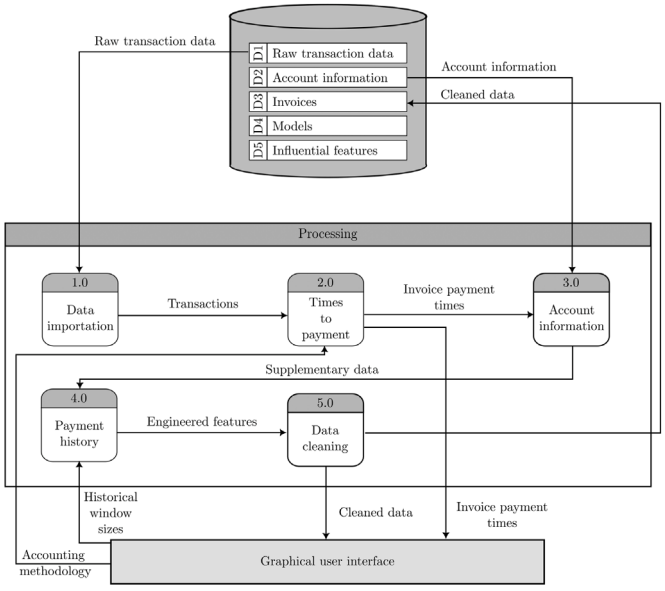


Fig. 2. Level-1 DFD of the processing component.

The central functional unit comprises three subcomponents, a *processing* component, a *modelling* component, and an *analysis* component. A *graphical user interface* (GUI) component is also employed to facilitate the exchange between commands issued by the user and information retrieved from the central functional component.

In the remainder of this section, each of the aforementioned components and subcomponents is described in some detail. The underlying processes of the subcomponents are presented with reference to lower-level DFDs aimed at elucidating the data that flow between the various framework modules.

3.1. The processing component

The processing component consists of five modules numbered 1.0 to 5.0, as illustrated in the Level-1 DFD in Fig. 2. The input data to the processing component comprises two data sets, one representing transactions associated with accounts that resulted in AR and the other containing accompanying supplementary data of the accounts considered. The focus in Modules 2.0 and 3.0 falls on the transformation of transaction data and supplementary data to a format that is suitable for analysis, while Modules 4.0 and 5.0 are responsible for engineering features and cleaning both the transaction and supplementary data sets, and transforming them into formats suitable for modelling.

The first step of the processing component involves importing the relevant data file. The user specifies the names and locations of the data files to be imported, after which the data types of the associated attributes are automatically identified. The raw transaction data are subsequently modified to create two separate data sets, representing payments and invoices, respectively.[2] Once separated, the cumulative debit amount (in the case of the invoice data set) and cumulative credit amount (in the case of the payment data set) are calculated per account and for transactions ordered chronologically. This data transformation lays the foundation for further analyses of time-to-event data in subsequent modules.

Once the data have been grouped according to the granularity mentioned above, the time to payment of each invoice can be determined

[2] For the sake of simplicity, it is recommended that credit notes and/or journal entries are not included in the raw transaction data set, but are instead accounted for outside of the transaction data set.

during the second step of the MIPP framework. In order to ensure that the framework remains generic and applicable in various problem domains, the accounting principles in both the *balance-forward* and *open-item* methodologies, are supported.

Balance-forward accounting. The theory that underpins balance-forward accounting is that payments associated with an account are allocated to the oldest debt associated with an account first. A critical intermediate process is, therefore, first required to identify the oldest debt associated with an account. This involves creating a debtors' ageing report in which the debt associated with each account is summarised according to the 'age' of the debt or the duration over which an invoice is past its due date. Once determined, a search algorithm may be constructed, as proposed by Schoonbee et al. (2022), to determine the time of payment for each invoice. Alternatively, the time to payment of each invoice may be determined by matching the cumulative debit amount for each invoice contained in the invoice data set, with the cumulative credit amount for all payments contained in the payment data set. Put differently, an invoice is only paid in full once an account holder has paid an amount equivalent to or larger than the cumulative outstanding debit amount at the time of invoicing. The payment resulting from the invoice is then considered as the first payment that meets this criterion.

Open-item accounting. The open-item accounting methodology is based on the notion that invoices remain unpaid or 'open' and that payments are made to settle specific invoices. The intermediate process of determining the oldest debt associated with an account is therefore not required and instead the payment date of each invoice is determined by matching each invoice contained in the invoice data set with its corresponding payment in the payment data set.

Once the payment dates are known for all invoices, the supplementary data stored in Data Store D2 are merged with the transaction data in Module 3.0. During this process, a primary key and a foreign key field are identified in the respective data sets, which are used to facilitate the join. This join is considered a *right join*, during which all the records in the transaction data set corresponding to a primary key are removed which do not match any foreign key in the supplementary data set.

The merged data set is then enhanced during a process of feature engineering in Module 4.0. This step involves generating new features based on historical customer payment behaviour, resulting in a data set that encapsulates both recent and earlier behaviour. The data set containing the newly engineered features is finally subjected to a process of data cleaning, in which the aim is to remove errors and inconsistencies from the data. The final processed data set is stored in Data Store D3, which is subsequently accessed by other components of the framework.

3.2. The modelling component

The modelling component is illustrated schematically in Fig. 3. This component consists of seven modules, which are numbered 6.0 to 12.0 in continuation of the module numbering in the processing component. Modules 6.0–8.0 are aimed at the development of relevant machine learning or survival analytic models employed in pursuit of a resolution of the IPPP. Modules 9.0–12.0 subsequently facilitate enhancements, prediction, and evaluations of user-selected models generated in the preceding modules.

In the MIPP framework, the IPPP is modelled as a classification problem, in which the objective is to predict the expected time to payment associated with a particular invoice, adopting fixed-interval classes. The number of fixed-interval classes is specified by the user. If, for example, the user specifies that four prediction classes should be used, the class labels 1–30 days, 31–60 days, 61–90 days, and 91+ days are configured.

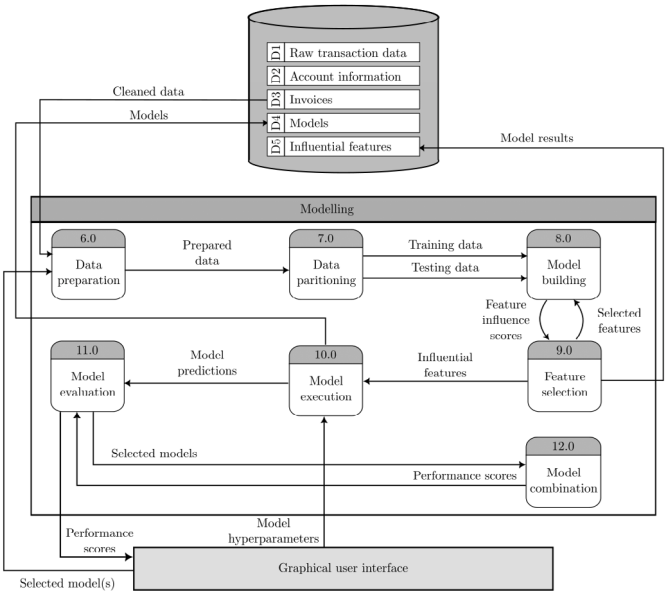


Fig. 3. Level-1 DFD of the modelling component.

The first step of the modelling component is to prepare the model data generated during execution of the processing component and the data stored in Data Store D3. Since right-censored invoice data are only relevant to models in the realm of survival analysis, it is important that the user specify the set of models to be configured from a given set. During this step, feature encoding is employed for categorical features and ranked, or ordinal features are normalised. The data partitioning module is invoked next to generate training and testing data sets for use during model development. In this context, the data are partitioned according to a *rolling time window* approach so as to minimise the chances of data leakage (Appel et al., 2019). An example of such a rolling time window approach would be to allocate all invoices dated before 2019 as part of the training set, while the remaining invoices fulfil the role of the testing set. In order to combat the problem of class imbalance, which frequently occurs in the IPPP, various class balancing techniques may be employed during execution of the data partitioning module, including undersampling, oversampling, *synthetic minority oversampling technique* (SMOTE) (Chawla et al., 2002), and hybrid approaches.

Appropriate prediction models are then configured based on the training data set supplied, after which feature influence scores are generated based on a process of statistical testing embedded in the parent model. In this context, the feature influence score represents the degree to which a particular feature influences the predicted outcome in the IPPP. After having identified features that would seem to be influential in terms of predictive power, the top n most influential features are stored in Data Store D5 and provided as input to the next module, the model execution module. In order to compensate for the trade-off between model performance and the computational burden associated with including all the features, the configured models are applied and tested utilising only the selected features.

The models' relative performances are measured next, by comparing their predicted invoice payment dates when applied to the testing data set. As mentioned earlier, the MIPP framework supports modelling the IPPP as a classification problem. Therefore, in the context of survival analysis models, a predicted time to payment is first determined by calculating the *restricted mean survival time* (RMST) (*i.e. the area under the survival curve*) (Irwin, 1949), after which the resulting time to payment is allocated to the predetermined class values. The model

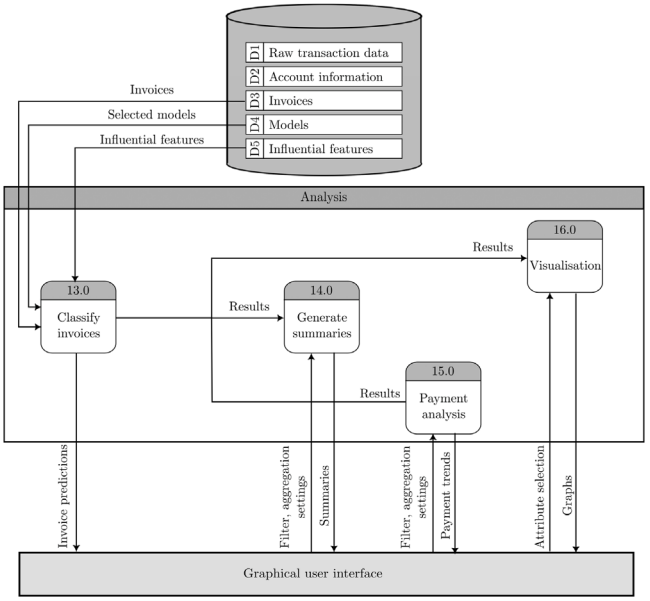


Fig. 4. Level-1 DFD of the analysis component.

performance is quantified by the *area under the receiver operating curve* (ROC-AUC) score (Bradley, 1997), classification accuracy (Gavrilov et al., 2000), and *F1*-score (van Rijsbergen, 1979) in the MIPP framework. The performance scores of the various models lay the foundation for further hyperparameter tuning in this module. During this process, the user of the framework supplies initial hyperparameter values for each model, after which a tuning process takes place according to a grid search approach, or some other automated technique such as the FLAML framework presented by Wang et al. (2021). Once tuned, the performances of the various user-selected models are summarised and presented to the user *via* the GUI component.

After having studied the performance comparison, the user may attempt to combine or *ensemble* various models by invoking either *sequential* or *parallel* ensemble methods (Breiman, 1996; Chen & Guestrin, 2016; Freund & Schapire, 1995; Schapire, 1990). A similar performance comparison is then relayed to the user, as before. This process is repeated iteratively until satisfactory performance has been achieved. Finally, the models and model results are stored in Data Stores D4 and D5, respectively.

3.3. The analysis component

A schematic representation of the final MIPP framework component, its analysis component, is presented in Fig. 4. This component comprises four modules, numbered 13.0–16.0, which are designed to facilitate an analysis and synthesis of the results generated by the modelling component for presentation to the user. The overarching objective pursued in the analysis component is to summarise the results generated by the processing component so as to be useful to the user during his or her decision-making process.

Module 13.0 facilitates the analysis of invoice payment predictions by combining the model data generated in Modules 1.0–5.0 and the predicted time to payment classes computed in Modules 6.0–12.0. Modules 14.0–16.0 are devoted to analysing the predicted payment classes associated with each of the invoices included in the data set transformed in Module 13.0. The focus in Module 14.0 is to generate an actionable summary of the predicted invoice classes. This module takes as inputs filter and aggregation settings specified by the user and delivers as output a summary of the results. The user may, for example, choose to view invoices together with their respective amounts for each of the predicted classes and per geographical area of the account holder.

In Module 15.0, payments are analysed in terms of historical trends and the probability of receiving payment over time. This entails creating visualisations of the average *days to payment* per month over time, as well as plotting survival curves for various stratified subsets of the data. In the former case, visualisations may take the form of line graphs, box plots or any alternative representation of the data. In the latter case, survival curves may be generated according to the Kaplan–Meier method (Kaplan & Meier, 1958) and visualised in the form of line graphs over time.

Further analysis is facilitated by Module 16.0. In this Module, the user is afforded the opportunity to select a subset of attributes from the classified invoices generated in Module 13.0, upon which a visualisation is returned depicting the relationship between the attributes selected, or the relationship between an attribute and the associated time to payment classes. By iteratively executing Modules 14.0–16.0 with varying input parameters, the user is able to explore the data set by viewing a summary of the predicted time to payment classes for a variety of invoices, trends and patterns in historical customer payment behaviour, and visualisations depicting the relationships between attributes or the associated time to payment classes. Based on these analyses, the user may identify trends or relationships that would seem to prevail in the data, after which appropriate business strategies may be developed to improve the overall AR collection efforts of the underlying business.

4. Case study experiments

In order to demonstrate the practicality of the proposed MIPP framework, an instantiation of the framework was implemented on a personal computer and applied to a real-world case study pertaining to a South African company which provides private education on primary and secondary school levels. In this case study, the *generic* components of the framework were populated subjectively with *specific* elements, so to demonstrate the practical applicability of an instantiation of the framework. The section opens with a brief background to the case study, after which the practical implementation of the various framework instantiation components is showcased.

4.1. Case study background

In the context of this case study, the management of AR collection is a particularly prevalent issue. This issue arises due to a large and diverse group of customer accounts monitored daily for the purpose of AR collections. Furthermore, increased interest rates pose a universal challenge in the South African consumer market for a large portion of the population, decreasing levels of disposable income in households, and increasing the likelihood of delinquency. Against this backdrop, if numerous accounts serviced by the company on which this case study is based were to become delinquent during the same month, collecting agents would be tasked with prioritising certain accounts for attention above others. To this end, the case study company adopts a conventional approach of contacting the holders of accounts with the largest outstanding amounts first. This approach is, however, problematic since it has been found that large amounts outstanding are not necessarily correlated with higher-risk clients (Appel et al., 2019). Moreover, since the case study company offers various product modes to its customers that vary in price, such a blanket approach might result in collection agents only prioritising delinquent accounts related to expensive products, while neglecting delinquent accounts from inexpensive products.

The current process of AR collection at the company may be described as a sequence of subprocesses that commence on the date on which a debtor receives an invoice and terminates when payment is received, or debt is written off as bad debt. All account holders

receive invoices on the first day of the month, which requires them to pay tuition fees in full for all pupils associated with their accounts for the current month. Should no payment be received during this month, account holders are contacted by the company's bursars in an attempt to collect outstanding debt. All accounts with invoices outstanding for more than 30 days, but for fewer than 90 days, are subsequently contacted from a centralised call-centre. During this phase of debt collection, the accounts with the oldest and largest outstanding amounts of debt are prioritised. Lastly, invoices that are outstanding for more than 90 days are forwarded to an external debt collector, which issues letters of demand to the respective account holders. Should an invoice still remain unpaid after more than two years, the company recognises that it is unlikely that such debt will be recovered and the debt is therefore classified as bad debt.

The problem considered in this case study is, therefore, to investigate to what extent an implementation of the MIPP framework can provide valuable business insight that might lead to an overall improvement in the collection efforts of the company in question. An additional outcome pursued in this case study is to present the company with information that leads to preventative actions, instead of the current reactive procedures that are in place.

4.2. Case study data processing

As mentioned in Section 3.1, the input to the processing component of the MIPP framework comprises two data sets. In this case study, the transactions data set contained 1 845 454 transactions associated with 72 036 distinct accounts. An additional client information data set was also imported which contained structured data attributes related to the accounts. Out of all the transactions recorded in the transactions data set, a total of 1 051 242 transactions were labelled as invoices, with the first invoice dated *2022/01/01* and the last invoice dated *2024/01/20*.

After successfully having imported the accompanying data files, a debtors' ageing report was constructed in tabular format for each account. The time to payment was subsequently calculated for each invoice contained in the ageing report by matching the cumulative debit amount in the invoice data set and the cumulative credit amount in the payment data set, as described in Section 3.1.

The transaction data set was subsequently merged with the client information data set during the following processing step. New features were generated next based on customer payment behaviour, domain knowledge, and the date of the relevant invoice. The features based on payment behaviour may be categorised as being either *lagging features i.e.* features that were created based on other feature values some number of time periods in the past or *time series features* (*i.e.* features created based on the average value of a feature over the past m periods). The features thus generated are summarised in Table 1.

Various data cleaning steps were applied to the data set in order to remove erroneous data values and invoices that do not apply to the MIPP framework. All invoices with opening credit balances less than or equal to the corresponding invoiced amounts were, for example, deleted since determining the dates of payment for such invoices was trivial. Finally, all invoices with missing or erroneous values were deleted from the data set to arrive at a final, prepared model data set.

Once the data had been processed and cleaned, an initial exploratory data analysis could commence with a view to gain a better understanding of the underlying characteristics of the data set. In order to gain an understanding of the time to payment for both censored and uncensored invoices, a *Kernel Density Estimation* (KDE) plot was first generated, as illustrated in Fig. 5.

In Fig. 5 it may be noted that a large majority of payments were received within the first few days after invoicing or around the 30th day after invoicing in the case of uncensored invoices (denoted by Censor = 0). This pattern was then repeated in 30-day intervals, which may be indicative of the fact that accounts included in this case study only settle their invoices after having received salary payment towards the end of the month. In the case of censored invoices (denoted by Censor = 1), a large majority of invoices were paid close to 30 days after invoicing or within the first few days after invoicing. Upon inspecting Fig. 5 visually, it may be noted that the probability distribution in this particular case study does not follow any well-known theoretical statistical distribution, but rather approaches a composite distribution consisting of several normal distributions centred around 30-day intervals.

Correlations between the various features and the number of days to payment were also analysed during the exploratory data analysis

Table 1
Features generated during the case study, based on payment behaviour, time series analysis, and the date on which a particular invoice was issued.

Category	Name	Description
Payment history (lagging)	DTP_1	The number of days between the due date and the payment date of the previous invoice issued to an account.
	DTP_2	The number of days between the due date and the payment date of the invoice issued to an account two invoices back ($n-2$).
	DTP_3	The number of days between the due date and the payment date of the invoice issued to an account three invoices back ($n-3$).
	DTP_4	The number of days between the due date and the payment date of the invoice issued to an account four invoices back ($n-4$).
Payment history (time series)	DTP_4_avg	The average number of days to payment of the previous four invoices.
	DTP_4_wavg	The weighted average number of days to payment for the previous four payments. The weights 0.4, 0.3, 0.2, and 0.1 were used for the variables DTP_1 to DTP_4, respectively.
	DTP_2_trend	The gradient between DTP_2 and DTP_1.
	DTP_3_trend	The gradient between DTP_3 and DTP_1.
Domain knowledge	MOD	The months of debt a particular account owes at the moment of invoicing, calculated as the total balance outstanding divided by the invoice value.
	DaysEnrolled	The number of days the particular account has been active.
	DSLP	The number of days since an account has made a payment.
	AgeRatio	The age ratio between the guardian and the eldest dependent child associated with the account, calculated as the age of the parent divided by the age of the child.
Dates	InvoiceMonth	The calendar month in which the invoice was issued.
	InvoiceQuarter	The quarter of the year in which the invoice was issued.

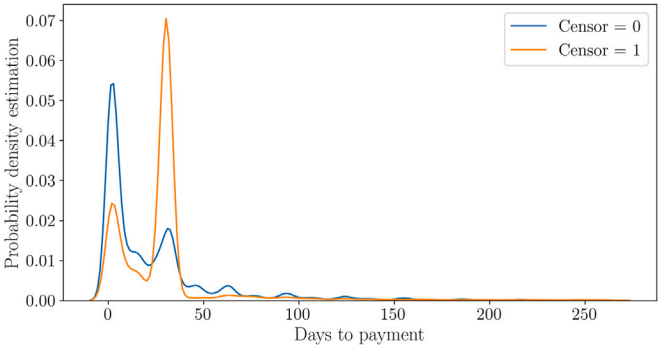


Fig. 5. KDE plot representing the probability density estimation for the times to payment of both censored and uncensored invoices.

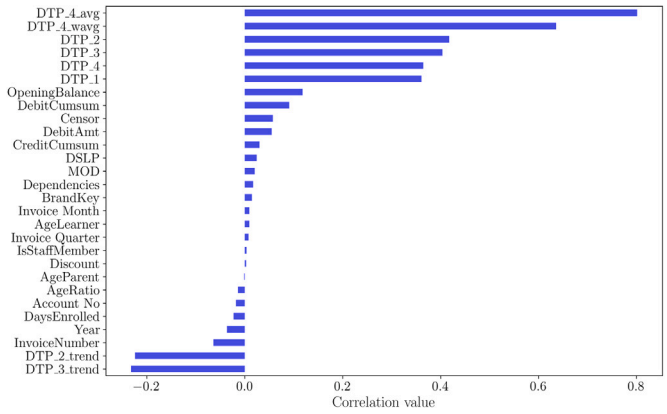


Fig. 6. Correlation scores between features and the days to payment variable.

phase and the results are illustrated visually in Fig. 6. By inspecting this visualisation, it is apparent that variables representing the recent payment behaviour of an account seem to be strongly correlated with the days to payment variable. Furthermore, there is a weak correlation between the features representing the demographic information of a particular account and the time to payment variable. Based on this finding, it is expected that the company in this case study might struggle to determine the affordability of its products for new account holders based on predetermined criteria and before any transactions have taken place.

4.3. Case study modelling

Once the model data had successfully been cleaned and stored in the database, further analyses could be performed according to the modelling component of the MIPP framework. These analyses were conducted in the programming language Python and leveraged various open-source packages that facilitate model development. After encoding the categorical features, the cleaned model data set employed in this case study contained 446 235 entries (or rows) and 35 attributes (or features), with each row representing a different invoice issued to a customer.

Out of the total number of invoices included in the case study, 67% were paid within 30 days, 23% were paid within 60 days, 6% were paid within 90 days, and 4% were paid after more than 90 days. A large class imbalance was therefore observed in the case study data, and various sampling techniques were employed to correct this imbalance. These techniques included undersampling, oversampling, SMOTE, and a hybrid approach (*i.e.* undersampling to some threshold and applying SMOTE to balance the classes). After having performed initial tests, it was found that the hybrid approach yielded the best model accuracy and so this approach was applied for all models going forward in the case study.

A rolling time window approach was adopted to apportion the invoices to a training and testing set. All invoices dated earlier than or on *2023/06/01* (70%) were taken as the training and testing set, while the invoices dated later than *2023/06/01* (30%) fulfilled the role of the validation set. This approach was adopted to prevent data leakage during model development.

4.3.1. Baseline model evaluation

During the initial model-building phase, five candidate machine learning models were developed and tested. These were a random forest (Breiman, 1996), the XGBoost algorithm (Chen & Guestrin, 2016), the k-nearest neighbour algorithm (Fix & Hodges, 1951), a decision tree, and the AdaBoost algorithm (Freund & Schapire, 2009). A feature selection procedure was executed for all of the models, except for the k-nearest neighbours algorithm, in which a combination of features was selected according to an embedded feature selection approach. During the first iteration of this approach, the feature importance scores of all features were computed, after which those with feature importance scores below a particular threshold value were removed. The threshold value was determined as the mean of all feature importance scores. By adopting this approach, a significant reduction in model complexity could be achieved while also retaining an acceptable performance level.

The selected features were then stored for each of the candidate models and were employed during the model execution phase of the modelling component. The relative performances of the various models were evaluated next according to the well-known metrics of classification accuracy, precision, recall, and *F1*-score.

4.3.2. Enhanced model description

During the model combination phase of the framework application, a novel sequential ensembling method was tested. A pseudo-code description of the algorithm, called the Survival Boost algorithm, may be found in Algorithm 1.

Algorithm 1: Survival Boost

Input : An array of invoices $\boldsymbol{X}$, a primary model P, an enhancement model E, and a set of m penalty values $\{p_0, \dots, p_m\}$.
Output : A set of predictions made for the invoices contained in $\boldsymbol{X}$, represented by an integer array Y.

```
BestConcordanceIndex ← 0;
for p ← p_0 to p_m do
    ConcordanceIndex ← FitSurvivalModel(X, p);
    if ConcordanceIndex ≥ BestConcordanceIndex then
        BestConcordanceIndex ← ConcordanceIndex;
        BestPenalty ← p;
EnhancementModel ← FitSurvivalModel(E, X, BestPenalty);
EnhancementFeatures ← GenerateFeatures(X,
 EnhancementModel);
InitialPrimaryModel ← TrainModel(P, EnhancementFeatures);
SelectedFeatures ← SelectFeatures(InitialPrimaryModel);
PrimaryModel ← TrainModel(P, SelectedFeatures);
Y ← MakePredictions(PrimaryModel);
return Y;
```

The first step of the algorithm is to select two candidate models, one of which has to be a classification machine learning model (hereafter referred to as the *primary* model, denoted by P) and the other a survival analysis model (hereafter referred to as the *enhancement* model, denoted by E). Both the enhancement model and the primary model are applied during the model development phase, after which only the primary model is applied during the feature selection, model execution, and model evaluation phases of the modelling component.

Once both candidate models have been selected, a set of penalty values $\{p_0, \dots, p_m\}$ that should be applied when fitting the enhancement model E is defined. The variable `BestConcordance`. `Index` is subsequently initialised to zero, after which the algorithm iterates over all remaining penalty values in $\{p_0, \dots, p_m\}$.

A `FitSurvivalModel` function is employed next so as to fit the enhancement model E to the invoice array $\boldsymbol{X}$, while applying a penalty value p_i. The resulting concordance index is recorded and compared with the current best concordance index value. If the value is greater than the current best concordance index value, the current concordance index value is taken as the new `BestConcordanceIndex` value and the variable `BestPenalty` is set to the current penalty score p_i.

After having determined the best penalty value, the function `FitSurvivalModel` is employed once again to fit the survival model to the invoice array $\boldsymbol{X}$, while applying a penalty value p_i. New features are subsequently generated by invoking a function `GenerateFeatures`, which takes as input a trained `EnhancementModel` and invoice array $\boldsymbol{X}$, and delivers as output a set of new features that are derived from the survival curve estimated for each invoice in $\boldsymbol{X}$.

An initial primary model is fitted by invoking a function `TrainModel`, after which a feature selection procedure is performed when applying a function `SelectFeatures`. Once the set of most influential features has been determined, the final `PrimaryModel` is trained in terms of the selected features, and the time to payment class for each invoice is predicted.

4.3.3. Enhanced model application

The enhancement model adopted for the specific purposes of the current study during the model combination phase was a Lasso-Cox model (Tibshirani, 1997). This model is often used in survival analytic studies in the presence of high-dimensional data sets in order to prevent overfitting the data set. The problem of overfitting is usually encountered in this domain, and for this reason it has been encouraged in the survival analysis literature to make use of sparsity norms, so as only to select those variables or features that best describe the outcome of the model (Friedman et al., 2001).

Suppose an observation is made for an individual j with the properties $(\boldsymbol{X}_j, \delta_j, y_j)$ in a survival analysis context, where $\boldsymbol{X}_j$ denotes the covariate vector, and δ_j denotes a binary parameter assuming the value 1 if instance j is censored, or 0 otherwise. Furthermore, let y_j denote the failure time experienced by individual j, which is taken as the observed time T_j in an uncensored case, or as the censored time C_j in a censored case. Let $\mathcal{R}_j$ be the set of all subjects that have not experienced the event of interest by time T_j, *i.e.* the risk set. The survival function is then expressed mathematically according to the basic Cox Proportional Hazards Model (Cox & Oaks, 1984) as

$$S(t, \boldsymbol{X}_i) = e^{-H_0(t)e^{\boldsymbol{X}_i\beta}} = S_0(t)^{e^{\boldsymbol{X}_i\beta}}, \tag{6}$$

where $H_0(t)$ and $S_0(t)$ denote the *cumulative baseline hazard function* and *baseline survival function*, respectively (Flynn, 2012). In order to estimate the coefficient vector $\boldsymbol{\beta}$, the negative partial *log-likelihood* function

$$L'(\beta) = -\sum_{j=1}^{N} \delta_j \left(X_j\beta_j - \log\left(\sum_{i \in \mathcal{R}_j} e^{X_i\beta_j} \right) \right) \tag{7}$$

is minimised. The Lasso-Cox model, therefore, is an extension of the basic Cox model, which simultaneously performs variable selection and coefficient estimation by shrinking certain coefficient values towards zero or exactly zero, thus improving the overall sparsity of the model (Tibshirani, 1996). This is achieved by applying a penalty value λ to the partial log-likelihood function, so that coefficient values are estimated as

$$\hat{\boldsymbol{\beta}} = \text{argmin} \left\{ \sum_{i=1}^{N} \left(y_i - \alpha - \sum_{j=1}^{P} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{P} \tau_j |\beta_j| \right\}, \tag{8}$$

where the weights $\tau_1, \ldots, \tau_P$ are adaptively determined based on the data, and α is estimated as $\hat{\alpha} = \bar{y}$ (the mean of $y_1, \ldots, y_N$).

Various penalty values were considered in order to determine the best performing penalty value for the Lasso-Cox model and the resulting concordance index value was evaluated. The results of these tests are illustrated graphically in Fig. 7. It may be noted in the figure that the best concordance index value was achieved for a penalty of value 10^{-2}.

The enhancement model was next fitted to the data, adopting the best performing penalty value, and new feature values were generated for each of the invoices and stored in the database. These generated feature values include the expected survival time (*i.e.* the area under the survival curve), survival function estimates at 30, 60, 90, and 120 days, the cumulative hazard function at 30, 60, 90, and 120 days, the partial hazards value, and the logarithm of the partial hazards value.

These new feature values were incorporated into the model data set utilised by the primary model so as to *enhance* the data set and provide potential insight into the primary model gained from the enhancement model. The primary model was finally retrained on the enhanced data set, after which the accuracy and feature importance scores were recaptured.

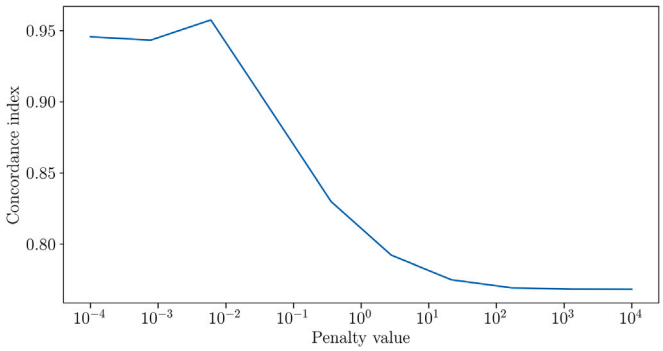


Fig. 7. The resulting concordance index when applying a penalty value to the Lasso-Cox model.

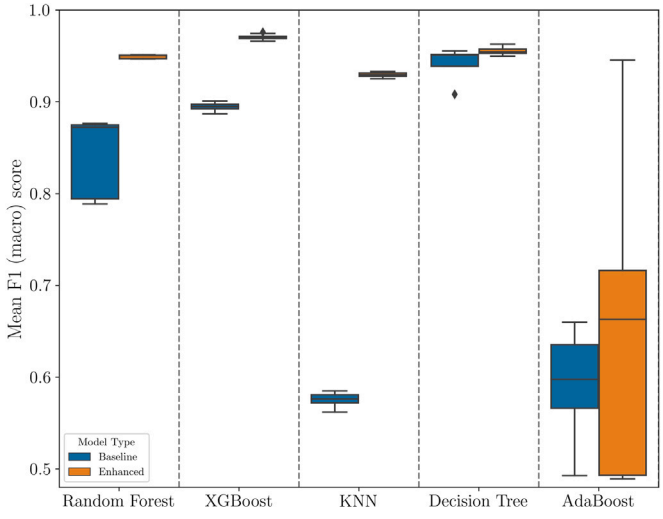


Fig. 8. The relative performance of the baseline and enhanced models, measured using the F1 macro score.

The relative performances of the five models are illustrated graphically in Fig. 8 and the aggregate F1 (macro)[3] scores for all the baseline and enhanced models are shown in Table 2. In this case, a random grid search was employed in conjunction with various hyperparameter value combinations for each model tested. A 5-fold cross validation approach was adopted to evaluate the performance of the various models, and the performance metric used in this case was the F1 macro score.

An interesting correlation was found amongst the models tested, in that a superior performance was obtained for all models in respect of the enhanced data set. The most significant improvement was observed for the k-nearest neighbours algorithm, where the average score was improved by 35.39%, while the smallest improvement in performance was noted for the decision tree models, where the average score was improved by only 1.46%. A detailed illustration of the performance improvements achieved for each model and hyperparameter combination is presented in Table A.4 in the Appendix.

Based on model performance, the enhanced XGBoost model was selected for further analysis in the analysis component of the framework. The model was first applied to a validation data set in which the model

[3] The F1 macro score refers to the scenario in which the F1 score is computed for each class independently and then averaged over all the classes.

Table 2
The performance scores achieved by the various models considered during execution of the MIPP framework modelling component when applied to the case study data.

Model	Baseline	Enhanced	Improvement
k-Nearest Neighbour	0.575328	0.929274	0.353945
Random Forest	0.846683	0.949374	0.102691
XGBoost	0.894198	0.970401	0.076204
AdaBoost	0.571850	0.643989	0.072139
Decision Tree	0.940660	0.955303	0.014643

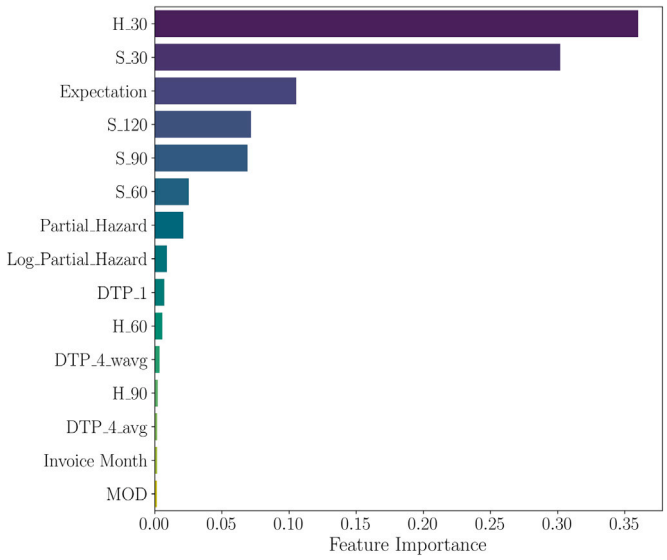


Fig. 9. Feature influence scores for the fifteen most influential features, as generated according to the enhanced XGBoost algorithm.

achieved an F1 (macro) score of 0.99. The top fifteen features in terms of feature importance scores were also analysed and are summarised in Fig. 9. Interestingly, the top eight most influential features were all generated by the enhancement model. The remaining important features are the engineered features — specifically those describing the payment history, as presented in Table 1.

After having evaluated the improvements achieved when applying the procedure outlined in the Survival Boost algorithm to conventional machine learning models, as illustrated in Fig. 8, a similar experiment was conducted using a more state-of-the-art methodology. As part of this analysis, an artificial neural network was trained on both the baseline data set (*i.e.* excluding features generated by a survival analysis model) and the enhancement data set (*i.e.* including features generated by a survival analysis model). Similar to the performance improvements illustrated in Fig. 8, an enhancement in overall classification accuracy was observed when utilising the enhancement data set during the training process of the artificial neural network. Furthermore, fewer epochs were required to achieve this improved accuracy than in the artificial neural network trained on the baseline data set.

Overall, these findings suggest that the *enhancement* model improved the predictive power of the primary model, indicating that models from the realms of survival analysis and machine learning hold potential to be ensembled in a sequential fashion with a view to achieving improved results. This concept was illustrated in the current case study by means of two types of analyses. First, the overall classification accuracy was shown to improve when utilising the enhancement data set in both conventional machine learning models, as illustrated in Fig. 8, and recent state-of-the-art machine learning models, as illustrated in Fig. 10. Secondly, the features generated by the enhancement model ranked among the top six most important features according to the feature influence scores, as illustrated in Fig. 9. Furthermore,

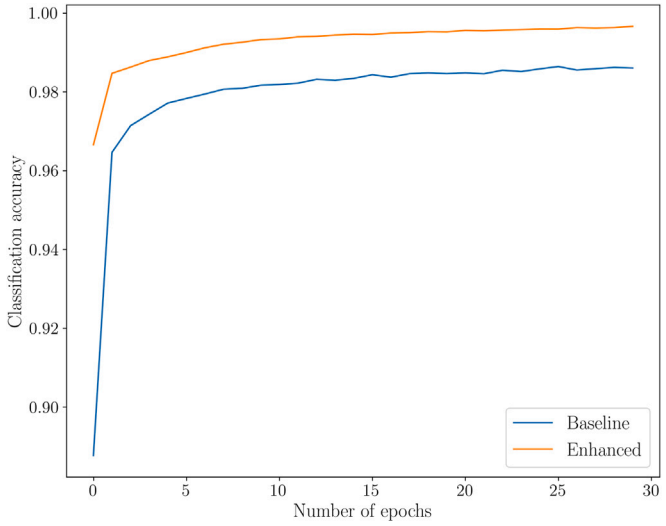


Fig. 10. The relative classification accuracy achieved by the baseline and enhanced artificial neural network per epoch.

Table 3
The predicted time to payment classes and the corresponding accumulated values of the invoices issued during November 2023, grouped according to geographical areas of customers. (One South African Rand (R) was worth approximately 5.2 US cents at the time of writing this paper.).

Region	30 Days	60 Days	90 Days	91+ Days	Total
Gauteng	R 38.08M	R 7.57M	R 2.24M	R 0.0M	R 47.89M
Western Cape	R 16.71M	R 3.80M	R 1.11M	R 0.0M	R 21.62M
KwaZulu-Natal	R 7.14M	R 2.16M	R 0.60M	R 0.0M	R 9.90M
Mpumalanga	R 4.12M	R 0.74M	R 0.36M	R 0.0M	R 5.22M
North West	R 2.21M	R 0.43M	R 0.10M	R 0.0M	R 2.73M
Northern Cape	R 1.32M	R 0.50M	R 0.08M	R 0.0M	R 1.90M
Limpopo	R 1.27M	R 0.20M	R 0.07M	R 0.0M	R 1.54M
Free State	R 0.76M	R 0.20M	R 0.10M	R 0.0M	R 1.06M

the high feature importance scores achieved for features capturing the payment history of customers reinforces a finding reported by various authors (Appel et al., 2019; Nanda, 2018; Smirnov, 2016), namely that historical payment behaviour enhances the predictive performance of models when applied to instances of the IPPP.

4.4. Case study results analysis

After having selected an appropriate model to deploy on the data set, the analysis phase commenced. Module 15.0 of the analysis component was executed and a summary was generated, as shown in Table 3. In this summary, the total number of invoices corresponding to various geographical areas were indicated, as were the numbers of invoices predicted to be paid within the fixed-interval classes. The purpose of this summary was to aid the case study company in managing its cash flow risks.

Module 16.0 was subsequently executed and the results in Fig. 11 were generated. This graphical illustration contains the respective survival curves for the geographical areas listed in Table 3. In this visualisation, the likelihood is presented that payment will be received for an invoice at a specific time, given that no payment had yet been received up to that point in time. More specifically, customers located in the Western Cape and KwaZulu-Natal provinces of South Africa were predicted to be *more likely* to pay their full invoiced amounts at 50 days than customers located in the other geographical areas. Such an observation could, for instance, be considered in conjunction with the values shown in Table 3, in order to adjust collection strategies, cash flow projections, bad debt provisions, and other business rules.

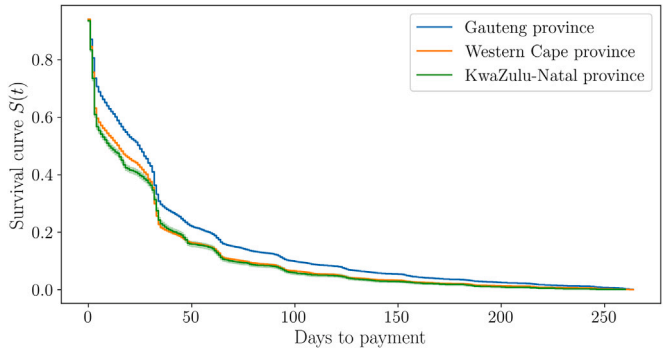


Fig. 11. Survival curves for the top three geographical areas in terms of invoiced value listed in Table 3.

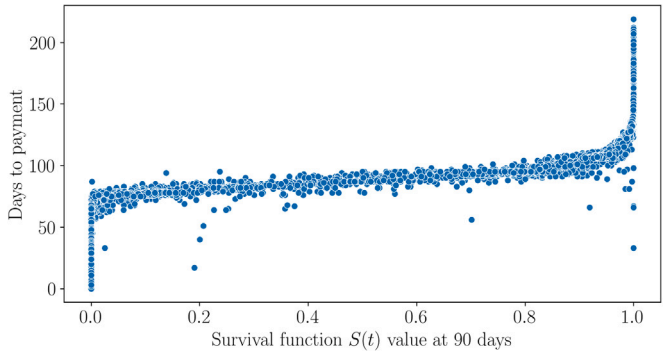


Fig. 12. The relationship between the enhancement variable S_90, depicting the survival curve value $S(t)$ at 90 days, and the actual days to payment of the validation set.

Finally, the discovery of relationships between structured variables and the predicted time to payment was facilitated during the execution of Module 17.0 of the analysis component. As illustrated in Fig. 12, various plots of the predicted time to payment may be constructed. In this figure, the actual time to payment value (in days) is compared with the feature value S_90 generated by the enhancement model. From this illustration, it is evident that most invoices paid before 50 days have an almost zero probability of 'survival' at day 90, while all invoices paid after 100 days have an almost 100% probability of 'survival' at day 90. The visualisation illustrates the usefulness of the variable S_90 in respect of improving upon the classification accuracy of models when predicting extreme values.

Each module in the analysis component of the MIPP framework may be invoked iteratively to extract meaningful insights, which may translate to improved business strategies. In order to gain the most value from the analysis component, however, the user should be familiar with certain core concepts in the survival analytic literature and use the information gained from the various framework components to guide the analysis performed in other components.

5. Conclusion

In this paper, we proposed a generic framework for modelling invoice payment predictions, called the MIPP framework. In order to place the framework within the context of the existing literature, an overview of similar research was first presented. The shortcomings of the existing approaches were subsequently outlined, including the limited information provided for the resolution of the IPPP in the case of reoccurring invoices, a lack of guidance in the form of a clear model development process, and a lack of generalisability and flexibility when analysing model results. A high-level overview of the MIPP framework was then proffered, after which each of the components of the framework was described in turn. The underlying processes of the various components were presented with reference to DFDs aimed at elucidating the data flows between the various modules.

An illustrative case study based on a real-world company in Southern Africa was presented next, so as to demonstrate the potential value and flexibility of the MIPP framework. The processing steps required to transform the data were recounted first, including the process of computing the time to payment by employing a search algorithm or debtors' ageing report. Some of the processes of the modelling component were demonstrated next, illustrating the application of prediction models to a particular case study. A novel algorithm, called the *survival boost* algorithm, was also introduced and its performance was illustrated in the context of enhancing the performance of other standard models. The working of the analysis component was demonstrated next, which included a summary of the predicted invoice values for one month and corresponding survival curves for different geographical groups of customers. Lastly, a relationship was illustrated between feature S_90, denoting the survival curve value $S(t)$ at 90 days, and the actual days to payment of each invoice included in the validation set.

5.1. Research contributions

Although there is an extensive body of literature in the domain of credit scoring, there is limited research addressing the IPPP as a classification problem. The MIPP framework proposed in this paper addresses the shortcomings in the existing literature in three fundamental ways. First, the framework was designed to guide the user through the entire process of preparing transaction data with a view to facilitate a resolution of the IPPP in the case of reoccurring invoices, where customer payments are allocated to the oldest debt first. While much of the existing literature describes the data used during model development, very little of the existing work describes the required data transformation process from transactions to invoice predictions. Furthermore, existing approaches in the literature are applied in the context where payments are linked to specific invoices. In contrast, the MIPP framework facilitates a more conservative approach according to which payments are allocated to the oldest debt first.

Secondly, the proposed MIPP framework was designed to be generic in nature. This was achieved by adopting a modular design approach so that modules included in the framework could be substituted, deleted, or modified. New findings in the literature may thus be incorporated into the framework without disrupting the overall execution of the framework. Furthermore, while most of the existing approaches in the literature focus on a select few very specific models, the user is guided through the process of developing a customised model in the MIPP framework. This identified lack of generalisability observed in the existing literature may be attributed to the relatively small body of work in this field and the particular application domains for which the existing approaches were developed.

Lastly, a new sequential ensembling method, called the *Survival Boost* algorithm, was proposed in this paper. According to this algorithmic approach, the performance of a machine learning algorithm may be enhanced through sophisticated feature engineering accomplished by a survival analysis model. We constructed a Lasso-Cox model based on the model data, after which various features were determined for each invoice in the data set. These feature values were then inserted as additional features into the original model data set and various machine learning classification models were applied to this *enhanced* data set. The primary and enhancement sub-algorithms of the survival boost algorithm may, however, be exchanged with other existing models in the respective survival analytic and machine learning literatures. It is important to note, however, that the Survival Boost algorithm should not be seen as a lone-standing algorithm whose performance may be

Table A.4
Improvements achieved in model performance.

Model	Param 1	Param 2	Param 3	Baseline	Enhanced	Improvement
AdaBoost	learning rate: 0.01	n estimators: 50	None	0.358982	0.945342	0.586360
KNN	n neighbours: 3	weights: uniform	None	0.561756	0.929995	0.368238
KNN	n neighbours: 5	weights: uniform	None	0.574504	0.931696	0.357191
KNN	n neighbours: 7	weights: uniform	None	0.577786	0.933094	0.355308
KNN	n neighbours: 3	weights: distance	None	0.571101	0.925097	0.353996
KNN	n neighbours: 5	weights: distance	None	0.581796	0.927191	0.345395
KNN	n neighbours: 7	weights: distance	None	0.585028	0.928569	0.343542
AdaBoost	learning rate: 1.0	n estimators: 150	None	0.492507	0.694129	0.201623
Random Forest	max depth: 10	min samples leaf: 1	n estimators: 100	0.788597	0.946728	0.158131
Random Forest	max depth: 10	min samples leaf: 1	n estimators: 200	0.789368	0.946855	0.157487
Random Forest	max depth: 10	min samples leaf: 1	n estimators: 300	0.791698	0.946844	0.155146
Random Forest	max depth: 10	min samples leaf: 2	n estimators: 200	0.793079	0.946782	0.153702
Random Forest	max depth: 10	min samples leaf: 2	n estimators: 300	0.793949	0.946806	0.152858
Random Forest	max depth: 10	min samples leaf: 3	n estimators: 300	0.794148	0.946971	0.152823
Random Forest	max depth: 10	min samples leaf: 3	n estimators: 200	0.794094	0.946877	0.152783
Random Forest	max depth: 10	min samples leaf: 3	n estimators: 100	0.793998	0.946725	0.152727
Random Forest	max depth: 10	min samples leaf: 2	n estimators: 100	0.794822	0.946481	0.151659
AdaBoost	learning rate: 0.01	n estimators: 100	None	0.566212	0.716076	0.149864
AdaBoost	learning rate: 1.0	n estimators: 100	None	0.597463	0.744300	0.146836
XGBoost	max depth: 10	min samples leaf: 3	n estimators: 100	0.886698	0.970910	0.084213
XGBoost	max depth: 10	min samples leaf: 1	n estimators: 100	0.886698	0.970910	0.084213
XGBoost	max depth: 10	min samples leaf: 2	n estimators: 100	0.886698	0.970910	0.084213
XGBoost	max depth: 10	min samples leaf: 2	n estimators: 200	0.892077	0.974241	0.082164
XGBoost	max depth: 10	min samples leaf: 3	n estimators: 200	0.892077	0.974241	0.082164
XGBoost	max depth: 10	min samples leaf: 1	n estimators: 200	0.892077	0.974241	0.082164
Random Forest	max depth: 20	min samples leaf: 3	n estimators: 100	0.868562	0.950416	0.081855
XGBoost	max depth: 10	min samples leaf: 3	n estimators: 300	0.895485	0.975748	0.080263
XGBoost	max depth: 10	min samples leaf: 2	n estimators: 300	0.895485	0.975748	0.080263
XGBoost	max depth: 10	min samples leaf: 1	n estimators: 300	0.895485	0.975748	0.080263
Random Forest	max depth: 20	min samples leaf: 3	n estimators: 200	0.870663	0.950385	0.079722
Random Forest	max depth: 30	min samples leaf: 1	n estimators: 100	0.871770	0.950817	0.079047
Random Forest	max depth: 20	min samples leaf: 3	n estimators: 300	0.871623	0.950498	0.078875
Random Forest	max depth: 20	min samples leaf: 1	n estimators: 100	0.872284	0.950661	0.078376
Random Forest	max depth: 20	min samples leaf: 1	n estimators: 300	0.873097	0.950887	0.077790
Random Forest	max depth: 30	min samples leaf: 3	n estimators: 100	0.873334	0.950322	0.076988
Random Forest	max depth: 20	min samples leaf: 2	n estimators: 200	0.873756	0.950589	0.076833
Random Forest	max depth: 20	min samples leaf: 1	n estimators: 200	0.874170	0.951003	0.076833
XGBoost	max depth: 30	min samples leaf: 3	n estimators: 100	0.889871	0.966446	0.076575
XGBoost	max depth: 30	min samples leaf: 2	n estimators: 100	0.889871	0.966446	0.076575
XGBoost	max depth: 30	min samples leaf: 1	n estimators: 100	0.889871	0.966446	0.076575

(*continued on next page*)

compared with those of other existing algorithms. Rather, the algorithm demonstrates the steps required to *enhance* the performance of such an existing algorithm through a process of sophisticated feature generation from a survival analytic model.

5.2. Research limitations and future work

As mentioned earlier, the scope of the MIPP framework was limited to classifying reoccurring invoices, where payments are made to the oldest invoice first. Although serving as a distinguishing factor of the framework from other work in the literature, this characteristic may also be considered a limitation. An alternative approach would be to allow the user to specify the debt collection policy that should be adopted during the processing component of the framework. The user might, for instance, decide whether the oldest debt should be paid off first or whether each payment should be allocated to a particular invoice. This enhancement would, however, require additional modules in the processing component of the framework, as well as additional data stores for recording transaction-to-payment allocations.

Furthermore, the MIPP framework was developed to guide the user through a generic model development process and therefore does not include model-specific steps required during model development. More specifically, the framework does not distinguish between the steps involved in creating a survival analytic model and those involved in creating a machine learning model. In the interest of practical applicability, it would be beneficial to include specific process steps for the different *types* of models so as to distinguish between the steps involved in these two modelling paradigms. A level 2 DFD could, for example, be provided to explode Module 9.0 into various child processes. Reference could be made to the development process of survival analytic models, including the selection of the censoring variable and specification of model parameters, among others. Similarly, a lower-level description could be provided in Module 12.0 to accommodate certain other performance evaluation metrics, such as the Brier score (Brier, 1950) and the concordance index (Harrell et al., 1982), which are only applicable to models from the realm of survival analysis.

CRediT authorship contribution statement

Willem Roux Moore: Conceptualisation, Methodology, Programming, Data curation, Writing – original draft. **Jan H. van Vuuren:** Supervision, Reviewing and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Table A.4 (*continued*).

Model	Param 1	Param 2	Param 3	Baseline	Enhanced	Improvement
Random Forest	max depth: 30	min samples leaf: 1	n estimators: 200	0.874843	0.950934	0.076092
Random Forest	max depth: 30	min samples leaf: 2	n estimators: 100	0.874400	0.950490	0.076089
Random Forest	max depth: 20	min samples leaf: 2	n estimators: 100	0.874617	0.950612	0.075996
Random Forest	max depth: 30	min samples leaf: 3	n estimators: 300	0.874816	0.950569	0.075753
Random Forest	max depth: 20	min samples leaf: 2	n estimators: 300	0.874912	0.950660	0.075747
Random Forest	max depth: 30	min samples leaf: 1	n estimators: 300	0.875435	0.951056	0.075621
Random Forest	max depth: 30	min samples leaf: 3	n estimators: 200	0.875584	0.950533	0.074948
XGBoost	max depth: 30	min samples leaf: 3	n estimators: 200	0.894710	0.969318	0.074608
XGBoost	max depth: 30	min samples leaf: 2	n estimators: 200	0.894710	0.969318	0.074608
XGBoost	max depth: 30	min samples leaf: 1	n estimators: 200	0.894710	0.969318	0.074608
Random Forest	max depth: 30	min samples leaf: 2	n estimators: 300	0.876481	0.950883	0.074402
Random Forest	max depth: 30	min samples leaf: 2	n estimators: 200	0.876347	0.950711	0.074364
XGBoost	max depth: 30	min samples leaf: 3	n estimators: 300	0.897280	0.971311	0.074030
XGBoost	max depth: 30	min samples leaf: 1	n estimators: 300	0.897280	0.971311	0.074030
XGBoost	max depth: 30	min samples leaf: 2	n estimators: 300	0.897280	0.971311	0.074030
XGBoost	max depth: 20	min samples leaf: 3	n estimators: 100	0.893347	0.965910	0.072563
XGBoost	max depth: 20	min samples leaf: 2	n estimators: 100	0.893347	0.965910	0.072563
XGBoost	max depth: 20	min samples leaf: 1	n estimators: 100	0.893347	0.965910	0.072563
XGBoost	max depth: 20	min samples leaf: 1	n estimators: 200	0.897678	0.969037	0.071360
XGBoost	max depth: 20	min samples leaf: 3	n estimators: 200	0.897678	0.969037	0.071360
XGBoost	max depth: 20	min samples leaf: 2	n estimators: 200	0.897678	0.969037	0.071360
XGBoost	max depth: 20	min samples leaf: 3	n estimators: 300	0.900633	0.970690	0.070057
XGBoost	max depth: 20	min samples leaf: 2	n estimators: 300	0.900633	0.970690	0.070057
XGBoost	max depth: 20	min samples leaf: 1	n estimators: 300	0.900633	0.970690	0.070057
Decision Tree	max depth: 10	min samples leaf: 1	None	0.908025	0.962638	0.054613
Decision Tree	max depth: 10	min samples leaf: 2	None	0.908114	0.962666	0.054552
Decision Tree	max depth: 10	min samples leaf: 3	None	0.907957	0.962451	0.054494
AdaBoost	learning rate: 1.0	n estimators: 50	None	0.635233	0.662874	0.027641
Decision Tree	max depth: 20	min samples leaf: 3	None	0.948812	0.955380	0.006568
Decision Tree	max depth: 30	min samples leaf: 3	None	0.950637	0.954624	0.003987
Decision Tree	max depth: None	min samples leaf: 3	None	0.950491	0.954229	0.003738
Decision Tree	max depth: 20	min samples leaf: 2	None	0.949378	0.952066	0.002688
Decision Tree	max depth: 20	min samples leaf: 1	None	0.952688	0.954435	0.001747
Decision Tree	max depth: None	min samples leaf: 2	None	0.950913	0.949803	−0.001110
Decision Tree	max depth: 30	min samples leaf: 2	None	0.950856	0.949629	−0.001227
Decision Tree	max depth: None	min samples leaf: 1	None	0.955092	0.952960	−0.002132
Decision Tree	max depth: 30	min samples leaf: 1	None	0.954961	0.952760	−0.002200
AdaBoost	learning rate: 0.01	n estimators: 150	None	0.566026	0.559966	−0.006061
AdaBoost	learning rate: 0.1	n estimators: 50	None	0.616249	0.489001	−0.127247
AdaBoost	learning rate: 0.1	n estimators: 100	None	0.654146	0.492815	−0.161330
AdaBoost	learning rate: 0.1	n estimators: 150	None	0.659835	0.491400	−0.168435

Appendix. Model performance results

Performance improvements achieved by the various models considered during execution of the modelling component, based on the case study data. The results are ranked in descending order of performance improvement achieved (see Table A.4).

References

Abdou, H. (2009). Genetic programming for credit scoring: the case of Egyptian public sector banks. *Expert Systems with Applications*, *36*, 11402–11417. http://dx.doi.org/10.1016/j.eswa.2009.01.076.

Appel, A., Oliveira, V., Lima, B., Maltaffi, G., Santana, V., & de Paula, R. (2019). Optimize cash collection: Use machine learning to predict invoice payments. (pp. 1–9). http://dx.doi.org/10.48550/arXiv.1912.10828, arXiv:1912.10828.

Bahrami, M., Bozkaya, B., & Balsicoy, S. (2020). Using behavioral analytics to predict customer invoice payment. *Big Data*, *8*, 25–37. http://dx.doi.org/10.1089/big.2018.0116.

Banasik, J., Crook, J., & Thomas, L. (1999). Not if but when will borrowers default. *Journal of the Operations Research Society*, *50*, 1185–1190. http://dx.doi.org/10.1057/palgrave.jors.2600851.

Bellotti, T., & Crook, J. (2009). Credit scoring with macroeconomic variables using survival analysis. *Journal of the Operational Research Society*, *60*, 1699–1707. http://dx.doi.org/10.1057/jors.2008.130.

Bradley, A. (1997). The use of the area under the ROC curve in the evaluation of machine learning algorithms. *Pattern Recognition*, *30*, 1145–1159. http://dx.doi.org/10.1016/S0031-3203(96)00142-2.

Breiman, L. (1996). Bagging predictors. *Machine Learning*, *24*, 123–140. http://dx.doi.org/10.1023/A:1018054314350.

Brier, G. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, *78*, 1–3. http://dx.doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

Cao, R., Vilar, J., & Devia Rivera, A. (2009). Modelling consumer credit risk via survival analysis. *Statistics and Operations Research Transactions*, *33*, 3–30, URL http://eudml.org/doc/43039.

Chawla, N., Bowyer, K., Hall, L., & Kegelmeyer, W. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, *16*, 21–357. http://dx.doi.org/10.1613/jair.953.

Chen, T., & Guestrin, C. (2016). XGBoost: A scaleable tree boosting system. In *Proceedings of the 22nd international conference on knowledge discovery and data mining*. San Francisco, CA.

Cheong, M., & Shi, W. (2018). Customer level predictive modelling for accounts receivables to reduce intervention strategies. In *Proceedings of the 14th international conference of data science*. Las Vegas, NV.

Cox, D. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society. Series B. Statistical Methodology*, *34*, 187–220. http://dx.doi.org/10.1080/01621459.2020.1713792.

Cox, D. R., & Oaks, D. (1984). *Analysis of survival data* (1st ed,). CRC Press.

Dirick, L., Claeskens, G., & Baesens, V. (2017). Time to default in credit scoring using survival analysis: A benchmark study. *Journal of Operations Research Society*, *68*, 652–665. http://dx.doi.org/10.1057/s41274-016-0128-9.

Durand, D. (1941). Risk elements in consumer instalment financing, (1st ed.). New York: National Bureau of Economic Research.

Firth, M. (1976). The management of accounts receivable. In M. Firth (Ed.), *Management of working capital* (pp. 53–71). Palgrove.

Fix, E., & Hodges, J. L. (1951). *Discriminatory analysis. Nonparametric discrimination: Consistency properties*: *Technical report 4*, Randolph Field, Texas: USAF School of Aviation Medicine.

Flynn, R. (2012). Survival analysis. *Journal of Clinical Nursing*, *21*, 2789–2797. http://dx.doi.org/10.1111/j.1365-2702.2011.04023.x.

Freund, Y., & Schapire, R. (1995). A decision-theoretic generalization of on-line learning and an application to boosting. In *Proceedings of the 2nd european conference on computational learning theory*. Barcelona.

Freund, Y., & Schapire, R. (2009). A short introduction to boosting. *The Journal of Japanese Society for Artificial Intelligence*, *14*, 771–780.

Friedman, J., Hastie, T., & Tibshirani, R. (2001). *The elements of statistical learning* (1st ed.). Springer.

Gavrilov, M., Anguelov, D., Indyk, P., & Motwani, R. (2000). Mining the stock market (extended abstract): Which measure is best? In *Proceedings of the 6th international conference on knowledge discovery and data mining*. Boston, MA.

Harrell, F., Califf, R., Pryor, D., Lee, K., & Rosati, R. (1982). Evaluating the yield of medical tests. *Journal of the American Medical Association*, *247*, 2543–2546. http://dx.doi.org/10.1001/jama.1982.03320430047030.

Henley, W. (1995). *Statistical aspects of credit scoring* (Ph.D. thesis), Milton Keynes: The Open University, http://dx.doi.org/10.21954/ou.ro.0000e061.

Irwin, J. (1949). The standard error of an estimate of expectation of life with special reference to expectation of tumourless life in experiments with mice. *Journal of Hygiene*, *47*, 188–189. http://dx.doi.org/10.1017/s0022172400014443.

Kaplan, E., & Meier, P. (1958). Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, *282*, 457–481. http://dx.doi.org/10.2307/2281868.

Klein, J. P. (2005). Censoring and truncation. In J. P. Klein, & M. L. Moeschberger (Eds.), *Survival analysis: techniques for censored and truncated data* (pp. 63–90). Springer Science and Business Media.

Martikainen, M. (2023). *Predicting late payment of sales using statistical learning methods*. Finland: University of Technology Lappeenranta, URL https://urn.fi/URN:NBN:fi-fe2023072791795.

Nanda, S. (2018). Proactive collections management: Using artificial intelligence to predict invoice payment dates. *Credit Research Foundation*, *1Q*, 1–22, URL https://www.crfonline.org/wp-content/uploads/2018/04/HighRadius-Proactive-Collections-Management-Using-AI-1Q2018-Journal.pdf.

Narain, B. (1992). Survival analysis and the credit granting decisions. In L. C. Thomas, J. N. Crook, & D. B. Edelman (Eds.), *Credit scoring and credit control* (pp. 109–121). Clarendon Press.

Peiguang, H. (2015). *Predicting and improving invoice-to-cash collection through machine learning*. Massachusetts Institute of Technology, URL http://hdl.handle.net/1721.1/99584.

PWC (2018). IFRS 9 impairment practical guide: Provision matrix. *In depth, 2018–3*, 1–3, URL https://www.pwc.ch/en/publications/2020/IFRS%209%20-%20Impairment%20-%20Provision%20Matrix%20-%20Practical%20Guide.pdf.

Rokach, L. (2010). *Pattern classification using ensemble methods*: *vol. 1*, World Scientific Publishing Company.

Rychnovsky, M. (2018). Survival analysis as a tool for better probability of default prediction. *Acta Oeconomica Pragensia*, *1*, 34–36. http://dx.doi.org/10.18267/j.aop.594.

Schapire, R. (1990). The strength of weak learnability. *Machine Learning*, *5*, 197–227. http://dx.doi.org/10.1023/A:1022648800760.

Schoonbee, L., Moore, W. R., & Van Vuuren, J. H. (2022). A machine-learning approach towards solving the invoice payment prediction problem. *The South African Journal of Industrial Engineering*, *33*, 126–146. http://dx.doi.org/10.7166/33-4-2726.

Smirnov, J. (2016). *Modelling late invoice payment times using survival analysis and random forest techniques*. Tartu: University of Tartu Faculty of Science and Technology.

Stepanova, M., & Thomas, L. (2001). PHAB scores — Proportional hazards analysis behavioural scores. *Journal of the Operational Research Society*, *41*, 1007–1016. http://dx.doi.org/10.1057/palgrave.jors.2601189.

Tater, T., Dechu, S., Mani, S., & Maurya, C. (2018). Prediction of invoice payment status in account payable business process. In *Proceedings of the 16th international conference on service-oriented computing*. Hangzhou.

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society. Series B. Statistical Methodology*, *58*, 267–288, URL https://www.jstor.org/stable/2346178.

Tibshirani, R. (1997). The lasso method for variable selection in the cox model. *Statistics in Medicine*, *16*, 385–395. http://dx.doi.org/10.1002/(sici)1097-0258(19970228)16:4%3C385::aid-sim380%3E3.0.co;2-3.

van Rijsbergen, C. (1979). *Information retrieval* (2nd ed.). Butterworths.

Wang, J. (2005). Functions of survival time. In J. Wang, & E. T. Lee (Eds.), *Statistical methods for survival data analysis* (pp. 8–18). John Wiley & Sons.

Wang, C., Qingyun, W., Weimer, W., & Zhu, E. (2021). FLAML: A fast and lightweight AutoML library. In *Proceedings of the 4th MLSys conference*. San Jose, CA.

Zeng, S., Mellville, P., Lang, C., Boier-Martin, L., & Murphy, C. (2008). Using predictive analytics to improve invoice-to-cash collection. In *Proceedings of the 14th ACM SIGKDD international conference of knowledge discovery and data mining*. Las Vegas, NV.

Zhou, Z. (2012). *Ensemble methods: Foundations and algorithms* (1st ed.). Chapman & Hall/CRC.